

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Prediction Market	Documentation quality	High	
Timeline	2026-05-19 through 2026-05-22	Test quality	High	
Language	Solidity	Total Findings	1	Fixed: 1
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review	High severity findings ⓘ	0	
Source Code	Polymarket/polymarket-v2 🔗 #07858ac 🔗	Medium severity findings ⓘ	0	
Auditors	- Jennifer Wu Auditing Engineer - Tim Sigl Auditing Engineer - Paul Clemson Auditing Engineer	Low severity findings ⓘ	0	
		Undetermined severity findings ⓘ	0	
		Informational findings ⓘ	1	Fixed: 1

Summary of Findings

The Combinatorial Module extends Polymarket V2 with multi-leg conjunction positions within the existing module framework. A combinatorial condition is a canonical leg array composed of Binary conditions, NegRisk conditions, or both, with up to 50 legs. Each array maps to a deterministic condition ID (`moduleId = 3`) and settles with YES/NO complement semantics. The module supports split/merge, condition-level refinements (`splitOnCondition` / `mergeOnCondition`), decomposition and recomposition (`extract` / `inject` , `convertToYesBasket` / `mergeFromYesBasket`), resolution-aware reduction (`compress`), terminal settlement (`redeem`), and single-leg interoperability (`wrap` / `unwrap`) with Binary and NegRisk positions.

The feature is integrated into Exchange through `matchOrdersAndPrepareCombinatorial()` , which prepares the supplied combinatorial legs and matches orders in the same operator-submitted transaction. This path follows the same operator execution model and fee/fill accounting as standard matching.

The review identified one informational finding and four auditor suggestions. **POL-COM-1** highlights a weaker condition-resolution invariant in `CombinatorialModule` , where `_getConditionPayout()` treats any non-empty result array as resolved, rather than enforcing the expected two-element payout structure. We recommend reviewing the operational considerations and addressing the identified issues and suggestions. Overall, code quality and test coverage are high, but we recommend adding test cases for edge scenarios, such as creating combinatorial positions from re-migrated legacy NegRisk events.

Fix-Review Update 2026-05-23:

During the fix review, the client addressed all issues and suggestions by either implementing fixes or acknowledging them.

ID	DESCRIPTION	SEVERITY	STATUS
POL-COM-1	Weaker Condition Resolution Invariant	Informational ⓘ	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

i Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
 3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
 1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
 2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

Repo: `https://github.com/Polymarket/polymarket-v2`

Included Paths: `src/modules/CombinatorialModule.sol`, `src/libraries/ModuleIds.sol`, `src/libraries/Ids.sol`, `src/exchange/Exchange.sol`, `src/bridge/abstract/BridgeBase.sol`

Extensions: `sol`

Operational Considerations

The following operational assumptions are made for combinatorial flows:

Dual-arity migration correlation must be handled operationally: NegRisk migration enrollment is keyed by structured `eventId` (including arity). If the same legacy event root is prepared under different condition counts, distinct namespaces can coexist and produce semantically correlated resolved legs (for example, synthetic Other in the smaller-arity namespace and appended winner in the larger-arity namespace). Combinatorial validation only enforces canonical leg structure and does not enforce legacy-root equivalence across arities. The client should enforce downstream RFQ and listing controls that reject leg sets mapping to the same legacy event root across different arities.

Combinatorial upgrade safety: Upgrade reviews should ensure that `MAX_LEGS` is not lowered in a new `CombinatorialModule` implementation, because existing positions with larger stored leg arrays may no longer be able to use paths that re-store derived leg sets (for example, `compress()` or `extract()`). Reviews should also treat `RESULT_DENOMINATOR` as a system-wide invariant across modules. If it is changed, historical result arrays written under the old denominator will be interpreted on the new scale, causing incorrect combinatorial payouts for already resolved conditions unless a coordinated migration handles those stored results.

Bridged combinatorial conditions must be prepared manually: `mintFromBridge()` does not check whether the bridged position belongs to a prepared combinatorial condition. Preparing the condition requires knowing the `conditionId` of each underlying leg. These positions remain unresolvable until the condition is prepared correctly.

Terminal settlement assumes all underlying legs can resolve on-chain: If any leg remains unresolved or unresolvable, `redeem()` remains non-terminal and `compress()` can only remove resolved legs, leaving residual exposure tied to unresolved legs.

Cross-module auth is an operational dependency for wrap/unwrap: `wrap` and `unwrap` depend on `PositionManager.setCrossModuleAuth` being correctly configured for `CombinatorialModule`. Misconfiguration can break these flows.

Multicall can strand balances: Users should ensure multicall bundles finish with no leftover collateral or position balances in `CombinatorialModule`. Stranded balances are not user-scoped inside the module and can be consumed by subsequent callers.

Market makers must correctly price combinatorial positions with correlated NegRisk conditions: There is no smart-contract-level restriction preventing a combinatorial condition from including two NO positions from the same NegRisk event. If market makers price those legs independently, quoted prices can diverge from the true probability of the combinatorial position resolving to the true value.

Key Actors And Their Capabilities

Core combinatorial flows are permissionless, including `prepareCondition`, `split / merge`, `splitOnCondition / mergeOnCondition`, `extract / inject`, `convertToYesBasket / mergeFromYesBasket`, `compress`, `redeem`, `wrap`, `unwrap`, and `multicall`, subject to token approvals/transfers and module validation constraints.

Combinatorial Module Owner

The owner can authorize UUPS upgrades for `CombinatorialModule` and can grant the admin role.

Combinatorial Module Admin

Admins can grant/revoke bridge-role access through `addBridge()` and `removeBridge()`.

Bridge Role (`_ROLE_3`) on Combinatorial Module

Bridge-role addresses can mint and burn combinatorial positions through `mintFromBridge()` and `burnFromBridge()`.

PositionManager Admin (Integration Prerequisite)

`wrap / unwrap` rely on cross-module mint/burn permissions. `PositionManager` admins must correctly configure module registration and `setCrossModuleAuth` for combinatorial interoperability with underlying modules.

Findings

POL-COM-1 Weaker Condition Resolution Invariant

• Informational ⓘ Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: `5ea1406bfd6614352e01b9254edb35810477d505`.

File(s) affected: `src/modules/CombinatorialModule.sol`

Description: In the `CombinatorialModule` contract, the `_getConditionPayout()` function reads the resolution status and payout array of a leg from the underlying module via `BaseModule.getResult()`. If the returned array has a length of zero, the condition is treated as unresolved; otherwise, any non-zero length causes the condition to be treated as fully resolved. However, to align with the rest of the `CombinatorialModule` and the structural design of binary and negative risk modules, the payout array is expected to have a length of exactly two. Relying solely on `res.length == 0` for the unresolved state is a weaker invariant than explicitly validating `res.length != 2` to signal an unresolved condition.

Recommendation: Modify the resolution check in `_getConditionPayout()` to verify that the result array has a length of exactly two, and treat any other length as unresolved.

Auditor Suggestions

S1 Miscellaneous Improvements

Fixed

✓ Update

Marked as "Acknowledged" by the client.
Addressed in: `935446ff41c808d0eb76932bd6f18b09be6d8fe3`.
The client provided the following explanation:

Fixed 1 and 2. Acknowledging 3, the reason we opt to not do this is because `getConditionId` takes memory, the current encoding can operate directly on calldata.

File(s) affected: `src/modules/CombinatorialModule.sol`, `src/exchange/Exchange.sol`

Description: The following changes would improve the overall quality of the codebase:

1. **Fixed** In the `CombinatorialModule` constructor, assign `MAX_LEGS` at its declaration (e.g., `uint256 public constant MAX_LEGS = 50;`) instead of in the constructor body.
2. **Fixed** In `matchOrdersAndPrepareCombinatorial()` , validate that `combinatorialLegs` derive the same combinatorial conditionId as `takerOrder.tokenId.conditionId()` before executing the order match, instead of relying on unchecked operator-provided legs.
3. **Acknowledged** In `prepareCondition()` , derive the condition ID by calling `getConditionId(_legs)` instead of duplicating the encoding logic inline. This is behaviorally equivalent today, but using the helper would make the ID derivation easier to audit and reduce the chance of future drift if the encoding ever changes.

Recommendation: Consider applying the code improvements as described.

S2 Readability of the `Compressed` Event Could Be Improved

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `71b30ad6d33874d1ee0fc065502a1dede1410984` .

File(s) affected: `src/modules/CombinatorialModule.sol`

Description: The `Compressed` event emits the old and new position IDs as well as the original input amount, but it does not emit the amount of collateral redeemed via `compress()` (`collateralOut`) or the amount of the new position ID that was minted (`positionAmount`). Although these values could be inferred by directly tracking the position and collateral token mint events, providing the data in a single event increases simplicity.

Recommendation: Consider adding additional fields to the `Compressed` event to give off-chain indexers better clarity.

S3 Consider Guarding Zero-Amount

Acknowledged

i Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

No economic impact and our offchain indexing handles this. Skipping events in `mintFromBridge` on `amount=0` is fine since we'll track bridge events from our bridge directly in our offchain indexers.

File(s) affected: `src/modules/CombinatorialModule.sol`

Description: In `CombinatorialModule` , several operations can emit events with zero amounts. This differs from `CombinatorialModule.mintFromBridge` , where `_amount = 0` is treated as a no-op and `BridgePositionMinted` is not emitted.

Recommendation: Consider adding non-zero guards to these operations.

S4 Remove Redundant Unchecked Loop Increments

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `ed13560d784449e679f079cae54b1668d0a910bc` .

File(s) affected: `src/modules/CombinatorialModule.sol`

Description: Since Solidity 0.8.22, the compiler automatically applies unchecked arithmetic to eligible `for` loop counter increments when the condition is `counter < bound` , the types match, and the counter is not modified in the loop body. The project compiles with Solidity 0.8.34, so manual `unchecked { ++i; }` blocks in loop bodies are redundant in functions where those conditions hold. The following 11 functions contain 17 loops that can be refactored:

1. `convertToYesBasket()` — 1 loop.
2. `compress()` — 1 loop.
3. `burnFromBridge()` — 1 loop.
4. `_validateCanonical()` — 1 loop.
5. `_requireConditionNotPresent()` — 1 loop.
6. `_insertLeg()` — 3 loops.
7. `_removeLeg()` — 2 loops.
8. `_prepareYesBasketPositionIds()` — 2 loops.

- 9. `_getYesBasketPositionIds()` — 2 loops.
- 10. `_burnMany()` — 1 loop.
- 11. `_getPositionPayout()` — 2 loops.

Recommendation: Consider moving each loop increment into the `for` header as a prefix increment (`++i`).

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

Repo: `https://github.com/Polymarket/polymarket-v2`

- `d59...8e0 ./src/bridge/abstract/BridgeBase.sol`
- `301...422 ./src/exchange/Exchange.sol`
- `e92...095 ./src/libraries/Ids.sol`
- `848...91c ./src/libraries/ModuleIds.sol`
- `031...703 ./src/modules/CombinatorialModule.sol`

Test Suite Results

The test suite can be run with the following command:

```
forge build
forge test --match-path "src/modules/test/*.t.sol"
```

```
Ran 4 tests for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_getPayout
[PASS] test_fullPayout() (gas: 104878)
[PASS] test_partialPayout() (gas: 124854)
[PASS] test_revert_conditionNotResolved() (gas: 40360)
[PASS] test_revert_invalidOutcomeIndex() (gas: 101817)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 10.42ms (198.58µs CPU time)

Ran 1 test for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_migrateAmountsMismatch
[PASS] test_revert_migratePositions_operator_amountsMismatch() (gas: 28934)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 10.28ms (235.42µs CPU time)
```



```
Ran 3 tests for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_resolverModifier
[PASS] test_revert_reportResult_resolutionPaused() (gas: 52730)
[PASS] test_revert_reportResult_resolverPaused() (gas: 50583)
[PASS] test_revert_reportResult_wrongSender() (gas: 20846)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 10.44ms (734.58µs CPU time)

Ran 1 test for src/modules/test/BinaryMigration.t.sol:BinaryMigrationTest_aliasKeyRegression
[PASS] test_revert_resolveMigrationCondition_aliasOutcomeByte() (gas: 86991)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 10.50ms (973.17µs CPU time)

Ran 1 test for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_resolveConditionToNoPauses
[PASS] test_revert_resolveConditionToNo_resolutionPaused() (gas: 215035)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 10.60ms (236.29µs CPU time)

Ran 4 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_unwrap
[PASS] test_revert_unwrap_multiCondition() (gas: 241355)
[PASS] test_unwrap_noPosition() (gas: 253626)
[PASS] test_unwrap_yesPosition() (gas: 251923)
[PASS] test_unwrap_yesPosition_ofNCondition() (gas: 250756)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 10.70ms (803.63µs CPU time)

Ran 12 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_bridge
[PASS] test_burnFromBridge() (gas: 115384)
[PASS] test_burnFromBridge_batch() (gas: 165778)
[PASS] test_conditionCount() (gas: 13888)
[PASS] test_conditionCount_invalidEventId() (gas: 11060)
[PASS] test_mintFromBridge() (gas: 108681)
[PASS] test_mintFromBridge_thenRedeem() (gas: 362122)
[PASS] test_mintFromBridge_zeroAmount() (gas: 80744)
[PASS] test_onERC1155BatchReceived_normalTransfer() (gas: 160301)
[PASS] test_onERC1155Received_normalTransfer() (gas: 119229)
[PASS] test_reportResult_bridgeDoesNotRequireEventBridge() (gas: 181525)
[PASS] test_reportResult_bridgeIdempotent() (gas: 184438)
[PASS] test_revert_mintFromBridge_unauthorized() (gas: 65096)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 11.08ms (697.33µs CPU time)

Ran 4 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_upgrade
[PASS] test_revert_upgrade_incompatibleModuleId() (gas: 4437960)
[PASS] test_revert_upgrade_incompatibleUsdce() (gas: 5485956)
[PASS] test_revert_upgrade_unauthorized() (gas: 5483435)
[PASS] test_upgradeToAndCall_preservesState() (gas: 5627893)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 11.09ms (1.37ms CPU time)

Ran 1 test for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_getPayoutCoverage
[PASS] test_revert_getPayout_invalidOutcomeIndex() (gas: 79191)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.97ms (38.63µs CPU time)

Ran 3 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_resolverRole
[PASS] test_addResolver_thenReport() (gas: 206967)
[PASS] test_reportResult_idempotent() (gas: 186746)
[PASS] test_revert_unauthorized() (gas: 23702)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 3.21ms (193.83µs CPU time)

Ran 2 tests for src/modules/test/NegRiskMigration.t.sol:NegRiskMigrationTest_aliasKeyRegression
[PASS] test_revert_resolveConditionToNo_aliasOutcomeByte() (gas: 943864)
[PASS] test_revert_resolveMigrationCondition_aliasOutcomeByte() (gas: 944110)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 14.88ms (5.27ms CPU time)

Ran 1 test for src/modules/test/BinaryMigration.t.sol:BinaryMigrationTest_cantina10
[PASS] test_migratePositions_doesNotInvokeReceiverHook() (gas: 403940)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 4.35ms (587.96µs CPU time)

Ran 2 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_viaRouter
[PASS] test_horizontalMerge_viaRouter() (gas: 468984)
[PASS] test_horizontalSplit_viaRouter() (gas: 319120)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 4.13ms (244.75µs CPU time)

Ran 2 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_prepareEventCoverage
[PASS] test_constructor_zeroNegRiskAdapter() (gas: 5459851)
[PASS] test_getEventId_isDeterministic() (gas: 14252)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 5.13ms (174.71µs CPU time)
```

```
Ran 1 test for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_split
[PASS] test_split() (gas: 279726)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 5.98ms (93.46µs CPU time)

Ran 1 test for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_legacyPositionId
[PASS] test_getLegacyPositionId_nonMigrated() (gas: 14900)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.92ms (26.83µs CPU time)

Ran 2 tests for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_hasResult
[PASS] test_hasResult_false() (gas: 37404)
[PASS] test_hasResult_true() (gas: 101759)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 3.64ms (59.71µs CPU time)

Ran 4 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_upgrade
[PASS] test_revert_upgrade_incompatibleModuleId() (gas: 4446876)
[PASS] test_revert_upgrade_incompatiblePositionManager() (gas: 30397680)
[PASS] test_revert_upgrade_unauthorized() (gas: 4991303)
[PASS] test_upgradeToAndCall_preservesState() (gas: 5079918)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 7.80ms (2.76ms CPU time)

Ran 4 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_convert
[PASS] test_convert() (gas: 375756)
[PASS] test_convert_emitsPositionConverted() (gas: 377310)
[PASS] test_revert_invalidConditionIndex() (gas: 20958)
[PASS] test_revert_invalidEventId() (gas: 16718)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 8.17ms (5.18ms CPU time)

Ran 5 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_reportResult
[PASS] test_allNoResolvesSyntheticOther() (gas: 360732)
[PASS] test_finalYesOutcome() (gas: 355968)
[PASS] test_reportResult() (gas: 180070)
[PASS] test_revert_multipleYesOutcomes() (gas: 232672)
[PASS] test_revert_unauthorized() (gas: 23717)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 3.24ms (331.21µs CPU time)

Ran 1 test for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_merge
[PASS] test_merge() (gas: 390202)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.94ms (124.71µs CPU time)

Ran 1 test for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_initialize
[PASS] test_revert_cannotReinitialize() (gas: 22678)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.31ms (16.00µs CPU time)

Ran 1 test for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_reportResultCoverage
[PASS] test_revert_migratePositions_operator_arrayMismatch() (gas: 28957)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.85ms (27.54µs CPU time)

Ran 5 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_directHorizontal
[PASS] test_horizontalMerge_direct() (gas: 446412)
[PASS] test_horizontalSplit_direct() (gas: 266353)
[PASS] test_revert_convert_noPreTransfer() (gas: 111915)
[PASS] test_revert_horizontalMerge_noPreTransfer() (gas: 93754)
[PASS] test_revert_horizontalSplit_noPreTransfer() (gas: 141458)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 3.37ms (356.50µs CPU time)

Ran 2 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_wrap
[PASS] test_wrap() (gas: 239964)
[PASS] test_wrapUnwrap_roundtrip() (gas: 287188)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 4.23ms (208.58µs CPU time)

Ran 4 tests for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_upgrade
[PASS] test_revert_upgrade_incompatibleModuleId() (gas: 5479878)
[PASS] test_revert_upgrade_incompatibleUsdce() (gas: 4439708)
[PASS] test_revert_upgrade_unauthorized() (gas: 4437198)
[PASS] test_upgradeToAndCall_preservesState() (gas: 4514179)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 8.57ms (1.69ms CPU time)

Ran 10 tests for src/modules/test/BinaryMigration.t.sol:BinaryMigrationTest_migrate
[PASS] test_migrate() (gas: 489383)
[PASS] test_migrate_asymmetricComplementaryBalances_sameCall() (gas: 482111)
[PASS] test_migrate_fromOperator() (gas: 498886)
[PASS] test_migrate_repeatedConditionEntries_clampsToCurrentBalance() (gas: 541491)
```

```
[PASS] test_prepareMigrationCondition_emitsMigrationConditionRegistered() (gas: 87425)
[PASS] test_revert_InvalidFromAddress() (gas: 355628)
[PASS] test_revert_alreadyResolved() (gas: 380470)
[PASS] test_revert_amountsLengthMismatch() (gas: 91447)
[PASS] test_revert_invalidArrayLength() (gas: 91448)
[PASS] test_revert_migrationNotRegistered() (gas: 198661)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 9.98ms (4.40ms CPU time)
```

```
Ran 5 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_resolveConditionToNo
[PASS] test_resolveConditionToNo_afterOneYes() (gas: 363210)
[PASS] test_resolveConditionToNo_emitsModuleAsResolver() (gas: 245286)
[PASS] test_revert_invalidConditionIndex() (gas: 25138)
[PASS] test_revert_invalidEventId() (gas: 19988)
[PASS] test_revert_reportResult_beforeAllYes() (gas: 114740)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 3.15ms (336.08µs CPU time)
```

```
Ran 2 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_getEventId
[PASS] test_getEventId() (gas: 13313)
[PASS] test_revert_invalidConditionCount() (gas: 28914)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 3.83ms (207.38µs CPU time)
```

```
Ran 7 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_extract
[PASS] test_extract() (gas: 477144)
[PASS] test_extract_fullDecomposition() (gas: 657702)
[PASS] test_revert_extract_requiresNoSide() (gas: 262914)
[PASS] test_revert_extract_unprepared() (gas: 189953)
[PASS] test_revert_extract_wrongToLength() (gas: 262978)
[PASS] test_revert_indexOutOfRange() (gas: 242295)
[PASS] test_revert_singleCondition() (gas: 221479)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 3.84ms (805.33µs CPU time)
```

```
Ran 8 tests for src/modules/test/NegRiskMigration.t.sol:NegRiskMigrationTest_prepareEvent
[PASS] test_maxLegacyConditionCount() (gas: 6219530)
[PASS] test_minLegacyConditionCount() (gas: 94933)
[PASS] test_prepareMigrationEvent_emitsLegacyEventIdInEventPrepared() (gas: 1535899)
[PASS] test_revert_alreadyPrepared() (gas: 1536513)
[PASS] test_revert_eventNotPrepared() (gas: 23320)
[PASS] test_revert_insufficientConditionCount() (gas: 21324)
[PASS] test_revert_legacyConditionCountTooLarge() (gas: 21309)
[PASS] test_revert_migrationNotSupported() (gas: 5633793)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 10.59ms (7.77ms CPU time)
```

```
Ran 5 tests for src/modules/test/BinaryMigration.t.sol:BinaryMigrationTest_operatorBatchMigrate
[PASS] test_batchMigratePositions() (gas: 551562)
[PASS] test_revert_amountsLengthMismatch() (gas: 94341)
[PASS] test_revert_invalidArrayLength() (gas: 94441)
[PASS] test_revert_positionIdsLengthMismatch() (gas: 94297)
[PASS] test_revert_unauthorized() (gas: 92043)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 6.65ms (3.09ms CPU time)
```

```
Ran 1 test for src/modules/test/CombinatorialModuleInvariant.t.sol:CombinatorialModuleInvariantTest
[PASS] test_handlerResolvePath() (gas: 158795)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 9.16ms (471.17µs CPU time)
```

```
Ran 3 tests for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_viaRouter
[PASS] test_merge_viaRouter() (gas: 386794)
[PASS] test_redeem_viaRouter() (gas: 415105)
[PASS] test_split_viaRouter() (gas: 278240)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 6.53ms (338.33µs CPU time)
```

```
Ran 1 test for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_getLegs
[PASS] test_getLegs() (gas: 228961)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.11ms (108.67µs CPU time)
```

```
Ran 4 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_getPayout
[PASS] test_fullPayout() (gas: 203901)
[PASS] test_partialPayout() (gas: 170392)
[PASS] test_revert_conditionNotResolved() (gas: 40480)
[PASS] test_revert_invalidOutcomeIndex() (gas: 200862)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 6.67ms (532.38µs CPU time)
```

```
Ran 3 tests for src/modules/test/BinaryMigration.t.sol:BinaryMigrationTest_prepareCondition
```



```
[PASS] test_revert_alreadyPrepared() (gas: 87155)
[PASS] test_revert_questionNotPrepared() (gas: 25168)
[PASS] test_revert_wrongOutcomeCount() (gas: 64316)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 2.29ms (70.25µs CPU time)

Ran 4 tests for src/modules/test/BinaryMigration.t.sol:BinaryMigrationTest_resolveMigrationCondition
[PASS] test_resolveMigrationCondition() (gas: 450781)
[PASS] test_revert_notResolvedOnLegacy() (gas: 93526)
[PASS] test_revert_resolutionPaused() (gas: 307636)
[PASS] test_unpauseAllowsResolution() (gas: 476757)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 3.27ms (1.19ms CPU time)

Ran 12 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_getPayout
[PASS] test_getPayout_noAllTrue() (gas: 336164)
[PASS] test_getPayout_noAnyFalse() (gas: 335948)
[PASS] test_getPayout_noFractionalComplement() (gas: 376296)
[PASS] test_getPayout_noLaterFalseBeforeAllResolved() (gas: 271700)
[PASS] test_getPayout_supportsManyFullPayoutLegs() (gas: 1974299)
[PASS] test_getPayout_supportsThirteenFractionalLegs() (gas: 1556777)
[PASS] test_getPayout_yesAllTrue() (gas: 336078)
[PASS] test_getPayout_yesAnyFalse() (gas: 331693)
[PASS] test_getPayout_yesFractionalProduct() (gas: 376179)
[PASS] test_getPayout_yesFractionalRoundsAtEnd() (gas: 372139)
[PASS] test_getPayout_yesLaterFalseBeforeAllResolved() (gas: 271743)
[PASS] test_revert_unresolved() (gas: 277362)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 5.56ms (2.53ms CPU time)

Ran 4 tests for src/modules/test/BinaryMigration.t.sol:BinaryMigrationTest_softViewAliasReverts
[PASS] test_getResult_unknownCanonicalReturnsEmpty() (gas: 13328)
[PASS] test_hasResult_unknownCanonicalReturnsFalse() (gas: 12697)
[PASS] test_revert_getResult_nonCanonical() (gas: 6183)
[PASS] test_revert_hasResult_nonCanonical() (gas: 6195)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 3.21ms (47.29µs CPU time)

Ran 1 test for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_initialize
[PASS] test_revert_cannotReinitialize() (gas: 22622)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.99ms (16.00µs CPU time)

Ran 2 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_hasResult
[PASS] test_hasResult_false() (gas: 37568)
[PASS] test_hasResult_true() (gas: 200848)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 7.77ms (235.54µs CPU time)

Ran 1 test for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_view
[PASS] test_getPositionId(bytes32,uint256) (runs: 256, µ: 393, ~: 393)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 30.32ms (27.42ms CPU time)

Ran 2 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_horizontalMerge
[PASS] test_horizontalMerge() (gas: 508514)
[PASS] test_revert_invalidEventId() (gas: 16661)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 3.99ms (184.79µs CPU time)

Ran 8 tests for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_bridge
[PASS] test_burnFromBridge() (gas: 97586)
[PASS] test_burnFromBridge_batch() (gas: 147963)
[PASS] test_mintFromBridge() (gas: 86400)
[PASS] test_mintFromBridge_thenRedeem() (gas: 240924)
[PASS] test_mintFromBridge_zeroAmount() (gas: 58496)
[PASS] test_onERC1155BatchReceived_normalTransfer() (gas: 138008)
[PASS] test_onERC1155Received_normalTransfer() (gas: 96959)
[PASS] test_revert_mintFromBridge_unauthorized() (gas: 42826)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 5.28ms (578.96µs CPU time)

Ran 3 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_inject
[PASS] test_inject() (gas: 534726)
[PASS] test_revert_inject_requiresNoSide() (gas: 217052)
[PASS] test_revert_inject_unprepared() (gas: 144083)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 5.82ms (2.68ms CPU time)

Ran 1 test for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_initialize
[PASS] test_revert_cannotReinitialize() (gas: 22645)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.81ms (12.96µs CPU time)
```

```
Ran 6 tests for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_direct
[PASS] test_merge_direct() (gas: 363600)
[PASS] test_redeem_direct() (gas: 392671)
[PASS] test_revert_merge_noPreTransfer() (gas: 86541)
[PASS] test_revert_redeem_noPreTransfer() (gas: 151206)
[PASS] test_revert_split_noPreTransfer() (gas: 92280)
[PASS] test_split_direct() (gas: 225741)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 3.99ms (434.67µs CPU time)

Ran 4 tests for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_reportResult
[PASS] test_reportResult() (gas: 77655)
[PASS] test_reportResult_idempotent() (gas: 83604)
[PASS] test_revert_invalidSum() (gas: 27778)
[PASS] test_revert_unauthorized() (gas: 20846)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 2.96ms (127.00µs CPU time)

Ran 3 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_merge
[PASS] test_merge() (gas: 315910)
[PASS] test_revert_merge_wrongModuleId() (gas: 17708)
[PASS] test_splitMerge_roundtrip() (gas: 303426)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 4.12ms (853.92µs CPU time)

Ran 2 tests for
src/modules/test/NegRiskMigration.t.sol:NegRiskMigrationTest_resolveConditionToNo_migration
[PASS] test_resolveConditionToNo_migratedLoser_redeemsLegacy() (gas: 1939864)
[PASS] test_revert_resolveConditionToNo_migratedLoser_legacyUnresolved() (gas: 1484570)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 20.53ms (15.52ms CPU time)

Ran 2 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_horizontalSplit
[PASS] test_horizontalSplit() (gas: 340500)
[PASS] test_revert_invalidEventId() (gas: 222554)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 6.19ms (525.08µs CPU time)

Ran 3 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_mergeFromYesBasket
[PASS] test_mergeFromYesBasket() (gas: 622735)
[PASS] test_revert_mergeFromYesBasket_requiresNoSide() (gas: 216964)
[PASS] test_revert_mergeFromYesBasket_unprepared() (gas: 143994)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 4.06ms (665.00µs CPU time)

Ran 3 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_canonicalization
[PASS] test_permutationYieldsSameId() (gas: 24929)
[PASS] test_revert_invalidModule() (gas: 14415)
[PASS] test_revert_invalidOutcomeIndex() (gas: 16602)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 3.90ms (216.00µs CPU time)

Ran 7 tests for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_resolver
[PASS] test_addResolver_thenReport() (gas: 106731)
[PASS] test_pauseResolution() (gas: 44700)
[PASS] test_pauseResolver() (gas: 44765)
[PASS] test_revert_reportResult_resolutionPaused() (gas: 52730)
[PASS] test_revert_reportResult_resolverPaused() (gas: 50561)
[PASS] test_unpauseResolution() (gas: 35067)
[PASS] test_unpauseResolver() (gas: 35345)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 6.41ms (208.54µs CPU time)

Ran 3 tests for src/modules/test/NegRiskMigration.t.sol:NegRiskMigrationTest_horizontalMerge
[PASS] test_horizontalMerge() (gas: 23324736)
[PASS] test_horizontalMerge_beforeResolution() (gas: 1341742)
[PASS] test_horizontalMerge_nonMigratedEvent() (gas: 427205)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 43.92ms (40.98ms CPU time)

Ran 12 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_compress
[PASS] test_compress_fractionalDustCannotExtractAfterRemainingLegResolves() (gas: 605376)
[PASS] test_compress_fractionalDustDoesNotLeakCollateral() (gas: 549264)
[PASS] test_compress_noPosition_allTrueWorthless() (gas: 364656)
[PASS] test_compress_noPosition_falseConditionMakesCollateral() (gas: 352406)
[PASS] test_compress_noPosition_fractionalConditionMintsCollateralAndReducedNo() (gas: 447445)
[PASS] test_compress_noPosition_trueConditionReduces() (gas: 380945)
[PASS] test_compress_yesPosition_allTrue_collateral() (gas: 404868)
[PASS] test_compress_yesPosition_falseConditionMakesWorthless() (gas: 318843)
[PASS] test_compress_yesPosition_fractionalConditionAfterUnresolvedScalesDown() (gas: 400392)
```

```
[PASS] test_compress_yesPosition_fractionalConditionScalesDown() (gas: 403954)
[PASS] test_compress_yesPosition_trueConditionRemoved() (gas: 382316)
[PASS] test_revert_nothingToCompress() (gas: 257745)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 5.13ms (1.96ms CPU time)

Ran 4 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_mergeOnCondition
[PASS] test_mergeOnCondition() (gas: 538001)
[PASS] test_revert_mergeOnCondition_parentMustBeYes() (gas: 197544)
[PASS] test_revert_mergeOnCondition_parentUnprepared() (gas: 143108)
[PASS] test_revert_mergeOnCondition_referenceMustBeConditionId() (gas: 6291)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 6.25ms (366.58µs CPU time)

Ran 11 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_bridge
[PASS] test_burnFromBridge() (gas: 81630)
[PASS] test_burnFromBridge_batch() (gas: 131037)
[PASS] test_mintFromBridge() (gas: 74004)
[PASS] test_mintFromBridge_thenRedeem() (gas: 257298)
[PASS] test_mintFromBridge_zeroAmount() (gas: 37977)
[PASS] test_onERC1155BatchReceived_normalTransfer() (gas: 120489)
[PASS] test_onERC1155Received_normalTransfer() (gas: 76297)
[PASS] test_revert_burnFromBridge_wrongModuleId() (gas: 150623)
[PASS] test_revert_mintFromBridge_unauthorized() (gas: 23735)
[PASS] test_revert_mintFromBridge_wrongModuleId() (gas: 22232)
[PASS] test_revert_redeemBridgedPosition_unprepared() (gas: 152635)
Suite result: ok. 11 passed; 0 failed; 0 skipped; finished in 4.48ms (641.29µs CPU time)

Ran 11 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_split
[PASS] test_prepareCondition_maxLegs() (gas: 1633505)
[PASS] test_revert_conflictingConditions() (gas: 19850)
[PASS] test_revert_duplicateCondition() (gas: 17503)
[PASS] test_revert_emptyConditions() (gas: 13850)
[PASS] test_revert_nonCanonicalOrder() (gas: 19708)
[PASS] test_revert_prepareCondition_tooManyLegs() (gas: 430637)
[PASS] test_revert_splitOnCondition_tooManyLegs() (gas: 1660518)
[PASS] test_revert_split_wrongModuleId() (gas: 18625)
[PASS] test_revert_wrongToLength() (gas: 21720)
[PASS] test_split() (gas: 150983)
[PASS] test_split_singleCondition() (gas: 177719)
Suite result: ok. 11 passed; 0 failed; 0 skipped; finished in 6.43ms (3.11ms CPU time)

Ran 5 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_convertToYesBasket
[PASS] test_convertToYesBasket() (gas: 554666)
[PASS] test_convertToYesBasket_singleCondition() (gas: 291754)
[PASS] test_revert_convertToYesBasket_requiresNoSide() (gas: 263096)
[PASS] test_revert_convertToYesBasket_unprepared() (gas: 190142)
[PASS] test_revert_convertToYesBasket_wrongToLength() (gas: 263580)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 3.64ms (558.08µs CPU time)

Ran 3 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_multicall
[PASS] test_multicall_canReturnCollateralFromRelatedNoPositions() (gas: 743510)
[PASS] test_multicall_twoSplits() (gas: 342945)
[PASS] test_revert_multicall_bubblesInnerError() (gas: 18267)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 4.10ms (814.25µs CPU time)

Ran 6 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_splitOnCondition
[PASS] test_revert_conditionAlreadyPresent() (gas: 199152)
[PASS] test_revert_splitOnCondition_parentMustBeYes() (gas: 198316)
[PASS] test_revert_splitOnCondition_parentUnprepared() (gas: 143898)
[PASS] test_revert_splitOnCondition_referenceMustBeConditionId() (gas: 6312)
[PASS] test_revert_splitOnCondition_wrongToLength() (gas: 243211)
[PASS] test_splitOnCondition() (gas: 478847)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 5.95ms (510.25µs CPU time)

Ran 10 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_redeem
[PASS] test_redeem_fractionalYesNoPairCannotExtractDust() (gas: 467371)
[PASS] test_redeem_noPosition_allTrue() (gas: 364207)
[PASS] test_redeem_noPosition_anyFalse() (gas: 404102)
[PASS] test_redeem_noPosition_fractionalComplement() (gas: 447597)
[PASS] test_redeem_noPosition_laterFalseBeforeAllResolved() (gas: 347289)
[PASS] test_redeem_yesPosition_allTrue() (gas: 407357)
[PASS] test_redeem_yesPosition_anyFalse() (gas: 360614)
[PASS] test_redeem_yesPosition_fractionalProduct() (gas: 447458)
```



```
[PASS] test_redeem_yesPosition_laterFalseBeforeAllResolved() (gas: 312633)
[PASS] test_revert_notFullyResolved() (gas: 314132)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 8.26ms (3.30ms CPU time)

Ran 2 tests for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_redeem
[PASS] test_redeem(bool) (runs: 256, μ: 416736, ~: 416736)
[PASS] test_redeem_emitsPositionRedeemed() (gas: 395524)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 59.57ms (57.27ms CPU time)

Ran 10 tests for src/modules/test/MigrationSnapshots.t.sol:MigrationSnapshots_Test
[PASS] test_MigrationSnapshots_binary_migrate_batch16() (gas: 747611)
[PASS] test_MigrationSnapshots_binary_migrate_batch2() (gas: 438560)
[PASS] test_MigrationSnapshots_binary_migrate_batch4() (gas: 478908)
[PASS] test_MigrationSnapshots_binary_migrate_batch8() (gas: 559705)
[PASS] test_MigrationSnapshots_binary_migrate_operator_batch4() (gas: 482280)
[PASS] test_MigrationSnapshots_negRisk_migrate_batch16() (gas: 5650625)
[PASS] test_MigrationSnapshots_negRisk_migrate_batch2() (gas: 1044495)
[PASS] test_MigrationSnapshots_negRisk_migrate_batch4() (gas: 1695188)
[PASS] test_MigrationSnapshots_negRisk_migrate_batch8() (gas: 3030984)
[PASS] test_MigrationSnapshots_negRisk_migrate_operator_batch4() (gas: 1698526)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 64.09ms (46.09ms CPU time)

Ran 2 tests for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_view
[PASS] test_getConditionId(bytes) (runs: 256, μ: 12078, ~: 12047)
[PASS] test_getPositionId(bytes32,uint256) (runs: 256, μ: 415, ~: 415)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 70.66ms (66.93ms CPU time)

Ran 6 tests for src/modules/test/NegRiskMigration.t.sol:NegRiskMigrationTest_resolveMigrationCondition
[PASS] test_resolveMigrationCondition() (gas: 20585020)
[PASS] test_revert_alreadyResolved() (gas: 1959196)
[PASS] test_revert_nonMigratedCondition() (gas: 1440015)
[PASS] test_revert_notResolvedOnLegacy() (gas: 1541112)
[PASS] test_revert_resolutionPaused() (gas: 1604027)
[PASS] test_unpauseAllowsResolution() (gas: 1817475)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 53.26ms (68.51ms CPU time)

Ran 1 test for src/modules/test/NegRiskMigration.t.sol:NegRiskMigrationTest_horizontalSplit
[PASS] test_horizontalSplit() (gas: 21589800)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 54.28ms (51.51ms CPU time)

Ran 6 tests for src/modules/test/NegRiskMigration.t.sol:NegRiskMigrationTest_resolveMigrationCounters
[PASS] test_countersUpdateOnNo() (gas: 1730851)
[PASS] test_countersUpdateOnYes() (gas: 1792080)
[PASS] test_finalizeMigration_allNo() (gas: 1770950)
[PASS] test_migrationThenResolveConditionToNo() (gas: 1772237)
[PASS] test_revert_lastConditionMustBeNo() (gas: 1682580)
[PASS] test_revert_twoYesLegacyResolves() (gas: 1836371)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 49.15ms (27.23ms CPU time)

Ran 2 tests for src/modules/test/NegRiskMigration.t.sol:NegRiskMigrationTest_view
[PASS] test_getConditionId(bytes32,uint8) (runs: 256, μ: 3176, ~: 3176)
[PASS] test_positionIds(bytes32,uint8,bool) (runs: 256, μ: 952, ~: 942)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 37.57ms (54.98ms CPU time)

Ran 10 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_pathIndependence
[PASS] testFuzz_extractPayoutProperty(uint96,uint256,uint256,uint256) (runs: 256, μ: 604141, ~: 605906)
[PASS] testFuzz_yesBasketPayoutProperty(uint96,uint256,uint256,uint256) (runs: 256, μ: 673341, ~: 674579)
[PASS] test_collateralReturnViaPartialPeel() (gas: 1204020)
[PASS] test_convertToYesBasket_matches_repeatedExtract() (gas: 771615)
[PASS] test_extractInjectRoundtrip() (gas: 533541)
[PASS] test_extractPayoutIdentity_boolean() (gas: 533101)
[PASS] test_extractPayoutWeakInequality_fractional() (gas: 598414)
[PASS] test_splitMergeValuePreservation() (gas: 621091)
[PASS] test_yesBasketPayoutIdentity_boolean() (gas: 596994)
[PASS] test_yesBasketPayoutWeakInequality_fractional() (gas: 667029)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 125.60ms (143.86ms CPU time)

Ran 4 tests for src/modules/test/NegRiskMigration.t.sol:NegRiskMigrationTest_migrate
[PASS] test_migrate() (gas: 20543645)
[PASS] test_revert_amountsLengthMismatch() (gas: 1537147)
[PASS] test_revert_conditionNotPrepared() (gas: 1438931)
[PASS] test_revert_invalidArrayLength() (gas: 1537165)
```


Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 48.98ms (54.60ms CPU time)

Ran 73 test suites in 251.30ms (1.02s CPU time): 297 tests passed, 0 failed, 0 skipped (297 total tests)

Code Coverage

The test suite's coverage of the in scope contracts can be obtained by running the following command:

```
forge coverage --no-match-coverage "test|EOA|mocks|legacy|dev|collateral|Quorum|Lz|Ccip|adapters"
```

File	% Lines	% Statements	% Branches	% Funcs
src/abstract/ERC1155TokenReceiver.sol	100.00% (7/7)	100.00% (6/6)	100.00% (0/0)	100.00% (3/3)
src/auth/InitializableRoles.sol	100.00% (18/18)	100.00% (9/9)	100.00% (0/0)	100.00% (9/9)
src/auth/Roles.sol	100.00% (32/32)	100.00% (16/16)	100.00% (0/0)	100.00% (16/16)
src/bridge/abstract/BridgeBase.sol	100.00% (123/123)	100.00% (137/137)	92.50% (37/40)	100.00% (22/22)
src/exchange/Exchange.sol	99.03% (508/513)	96.11% (593/617)	81.31% (87/107)	97.01% (65/67)
src/libraries/BridgePayloads.sol	100.00% (6/6)	100.00% (6/6)	100.00% (0/0)	100.00% (3/3)
src/libraries/Ids.sol	100.00% (43/43)	100.00% (49/49)	100.00% (2/2)	100.00% (19/19)
src/modules/BinaryModule.sol	100.00% (19/19)	100.00% (24/24)	100.00% (2/2)	100.00% (6/6)
src/modules/CombinatorialModule.sol	98.98% (390/394)	99.36% (465/468)	88.89% (104/117)	97.78% (44/45)
src/modules/NegRiskModule.sol	100.00% (100/100)	100.00% (124/124)	89.29% (25/28)	100.00% (16/16)
src/modules/abstract/BaseModule.sol	100.00% (47/47)	100.00% (47/47)	81.82% (9/11)	100.00% (10/10)
src/modules/abstract/OracleModule.sol	92.31% (24/26)	93.75% (15/16)	100.00% (6/6)	90.00% (9/10)
src/modules/migration/BaseMigrationMixin.sol	100.00% (80/80)	100.00% (98/98)	96.15% (25/26)	100.00% (10/10)
src/modules/migration/BinaryMigrationMixin.sol	96.67% (29/30)	92.00% (23/25)	85.71% (6/7)	100.00% (11/11)
src/modules/migration/NegRiskMigrationMixin.sol	96.00% (48/50)	94.64% (53/56)	87.50% (14/16)	91.67% (11/12)
src/oracle/OracleAggregator.sol	99.39% (163/164)	98.95% (189/191)	82.61% (57/69)	100.00% (19/19)

File	% Lines	% Statements	% Branches	% Funcs
src/oracle/abstract/OracleModuleBase.sol	100.00% (11/11)	100.00% (6/6)	100.00% (4/4)	100.00% (5/5)
src/oracle/libraries/OptimisticOraclePayoutLib.sol	100.00% (8/8)	100.00% (11/11)	100.00% (6/6)	100.00% (1/1)
src/oracle/mixins/Auth.sol	100.00% (12/12)	100.00% (6/6)	100.00% (0/0)	100.00% (6/6)
src/oracle/mixins/Pausable.sol	100.00% (8/8)	100.00% (5/5)	100.00% (2/2)	100.00% (3/3)
src/oracle/modules/OptimisticOracleModule.sol	100.00% (81/81)	98.92% (92/93)	92.31% (12/13)	100.00% (14/14)
src/oracle/modules/reporters/ChainlinkReporterModule.sol	100.00% (64/64)	100.00% (67/67)	100.00% (14/14)	100.00% (14/14)
src/positionManager/PositionManager.sol	94.29% (165/175)	94.67% (160/169)	85.29% (29/34)	100.00% (17/17)
src/routers/BridgeRouter.sol	100.00% (18/18)	100.00% (11/11)	100.00% (0/0)	100.00% (7/7)
src/routers/CtfRouter.sol	100.00% (35/35)	100.00% (42/42)	100.00% (3/3)	100.00% (6/6)
src/routers/Router.sol	100.00% (57/57)	100.00% (66/66)	100.00% (2/2)	100.00% (9/9)
src/utils/AutoRedeemer.sol	100.00% (98/98)	98.32% (117/119)	100.00% (18/18)	100.00% (6/6)
Total	98.87% (2194/2219)	98.11% (2437/2484)	88.05% (464/527)	98.63% (361/366)

Changelog

- 2026-05-22 - Initial report
- 2026-05-23 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp’s mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp’s team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1300 audits and secured over \$500 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Solana, Canton, Polymarket, PancakeSwap, Virtuals Protocol, Wayfinder, Lido, TrustWallet, Alchemy, World Economic Forum.

Quantstamp’s collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland

- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republic, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

